

MODERN KITCHEN: ARCHITECTURAL & TECHNICAL DOCUMENTATION

1. TITLE PAGE

Project Name: Modern Kitchen

Tagline: Bridging the Gap Between Professional Culinary Expertise and Home Cooking.

Prepared By: Aayush Thapa(Owner, Developer, System Architect - @Modern Kitchen)

Date: January 2026

Status: Production Ready

2. ABSTRACT

Modern Kitchen is a comprehensive full-stack marketplace ecosystem engineered to facilitate seamless interactions between professional chefs and home cooks. Facing a market where culinary guidance is often static and hiring professionals is fragmented, this platform introduces a streamlined, secure, and AI-enhanced solution. The technical foundation relies on **Next.js (App Router)** for a high-performance frontend/backend, **Clerk** for sophisticated identity management and role-based access control, **Prisma ORM** for type-safe data modeling, and **NeonDB** for a serverless, scalable PostgreSQL backend. A centerpiece of the innovation is **Riley**, a custom-prompted AI Chef Assistant providing 24/7 technical support. This documentation serves as a blueprint of the system's architecture, data flows, and deployment strategies.

3. PROBLEM STATEMENT

Technical Barrier: Home cooks often lack the technique required for advanced recipes, leading to wasted ingredients and poor results.

Market Fragmentation: There is no centralized, secure platform to hire local professional chefs for specialized home sessions or meal prep.

Lack of Real-Time Support: Traditional recipes are non-interactive; users cannot ask follow-up questions during the heat of the cooking process.

4. PROPOSED SOLUTION – MODERN KITCHEN

Modern Kitchen acts as a digital culinary bridge offering:

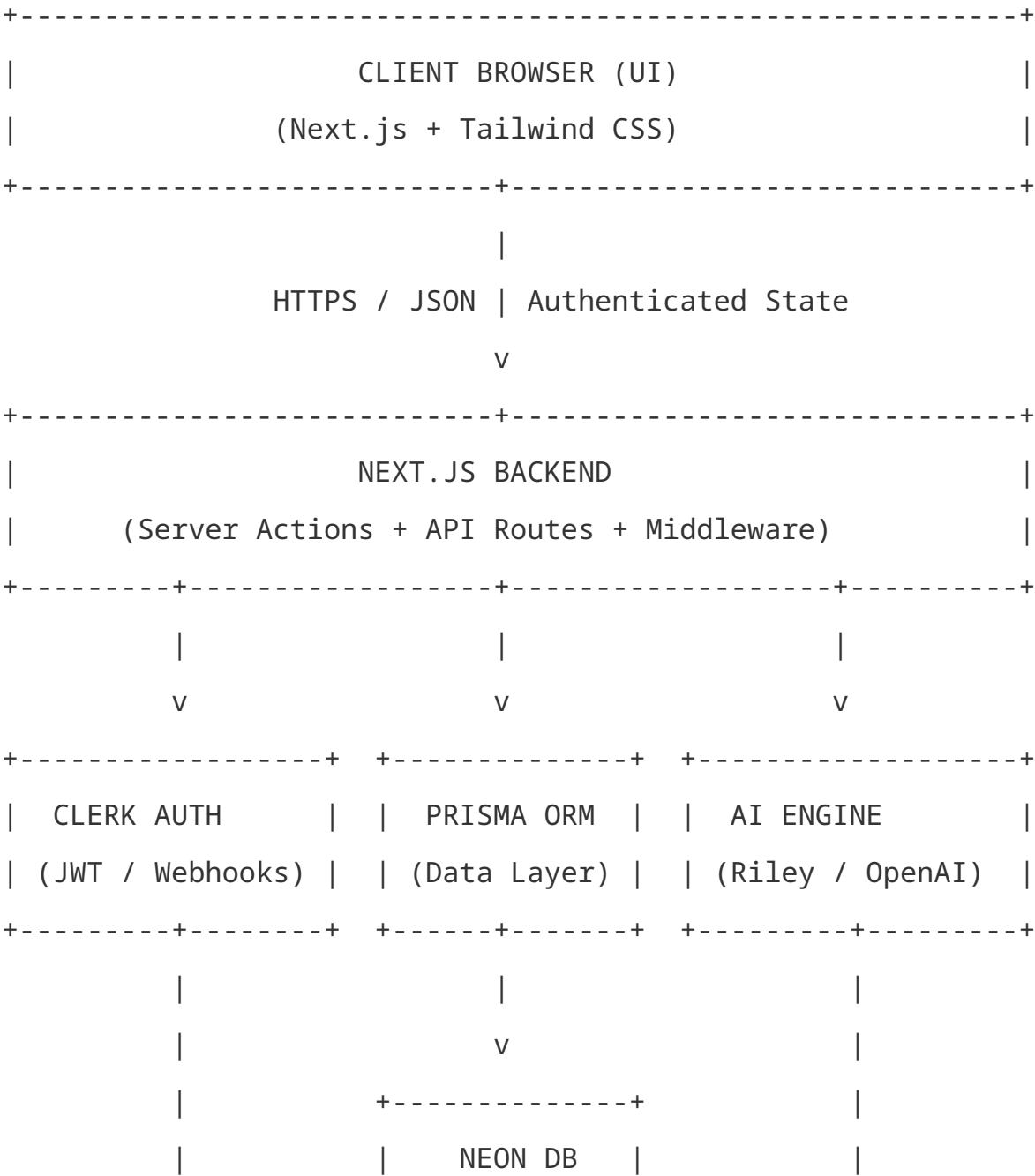
Role-Based Marketplace: A dual-interface system for Chefs to list services and Users to book them.

Riley AI: An integrated conversational assistant that acts as a real-time 'Sous Chef'.

Automated Trust: Secure payments, verified chef profiles, and community-driven reviews.

5. SYSTEM ARCHITECTURE

The platform utilizes a modern serverless architecture. The Next.js application acts as the central hub, communicating with Clerk for Auth, NeonDB for Persistence, and OpenAI for AI logic.



+----->| (PostgreSQL) |<-----+
+-----+

6. TECHNOLOGY STACK (DETAILED)

Frontend: Next.js 14+, React, Tailwind CSS, Lucide Icons.

Backend: Next.js Server Actions, Node.js.

Identity Management: Clerk (OpenID Connect, Webhooks).

Database: PostgreSQL (Serverless) via Neon.tech.

ORM: Prisma (Type-safe migrations and queries).

AI Assistant: Riley (GPT-4o API integration with custom system prompts).

Deployment: Vercel (Edge Functions enabled).

7. AUTHENTICATION & AUTHORIZATION (CLERK)

We leverage Clerk not just for login, but as our primary Authorization engine:

RBAC (Role Based Access Control): User roles (USER, ADMIN) are stored in Clerk's publicMetadata. One single ADMIN (OWNER-AayushThapa) USER-Clients.

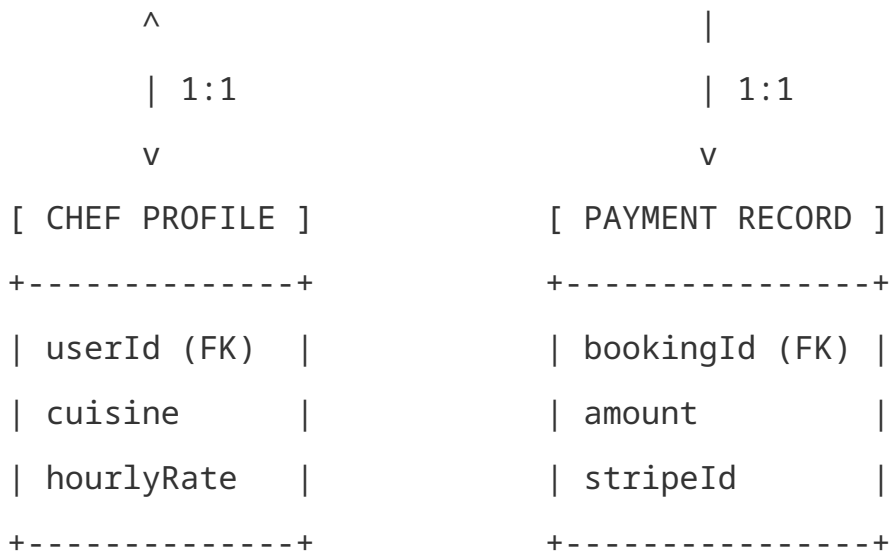
Middleware Protection: Next.js middleware intercepts requests to /admin to ensure the user has the 'ADMIN' email.

Session Security: Short-lived JWTs managed by Clerk with automatic rotation.

8. DATABASE DESIGN (PRISMA + POSTGRESQL)

The schema is designed for 3rd-party identity synchronization. We use the Clerk ID as the primary link.

[USER TABLE]		[BOOKING TABLE]
+-----+		+-----+
id (PK)	<----->	id (PK)
clerkId (U)	1:N	userId (FK)
email		chefId (FK)
role		date
+-----+		status
		+-----+



9. USER ROLES & RESPONSIBILITIES

Role	Core Responsibilities
Home Cook	Browse chefs, book sessions, consult Riley AI, manage session history.
Chef	Define availability, set pricing, manage booking requests, update culinary bio.
Admin	Marketplace moderation, user dispute resolution, analytics monitoring.
Riley (AI)	Technical recipe support, ingredient substitution advice, safety guidance.

10. FUNCTIONAL REQUIREMENTS

- Social Enrollment:** Google/GitHub login via Clerk.
- Marketplace Engine:** Searchable directory of chefs with filtering by cuisine.
- Dynamic Booking:** Calendar-based selection with conflict detection.
- AI Interfacing:** Chat window for recipe help without leaving the cooking dashboard.

11. NON-FUNCTIONAL REQUIREMENTS

- Scalability:** NeonDB auto-scaling allows the DB to scale to zero during idle time and burst during peak hours.

Latency: Edge-cached Clerk sessions ensure auth checks take < 50ms.

Data Integrity: Atomic Prisma transactions for booking processes.

12. BOOKING & SCHEDULING WORKFLOW

User selects a Chef and a 2-hour time slot.

System initiates a **Prisma Transaction** to check for overlaps.

If clear, a 'PENDING' booking is created.

Chef receives a notification (via Clerk/Email).

Upon Chef acceptance, the status updates to 'CONFIRMED' and payment intent is generated.

13. AI CHEF ASSISTANT – RILEY

Riley is implemented using a System Prompt to ensure domain-specific safety:

"You are Riley, a Senior Executive Chef. Provide concise, high-level culinary expertise. Prioritize safety (internal temperatures, knife skills). If a user asks for non-food advice, remind them you are 'in the kitchen' and cannot help with outside topics."

14. WEBHOOKS & EVENT HANDLING

Modern Kitchen uses **Svix** for Clerk webhook verification. This ensures our local NeonDB stays in sync with Clerk's identity store.

`user.created`: Triggers a Prisma upsert to create a profile in our DB.

`user.deleted`: Cleans up local profile data to ensure GDPR compliance.

15. SEQUENCE DIAGRAMS

Scenario: User Booking a Chef

User (Client)	Next.js (API)	Prisma/NeonDB	Clerk Auth
-- Select Chef -->			
	-- Verify Auth -->		
	<---- Valid -----		

```
|                                |-- Check Slots -->|                                |
|                                |<-- Available ----|                                |
|                                |                                |                                |
|                                |-- Create Book ---|                                |
|<-- Success -----|                                |
```

16. SECURITY & PRIVACY

- At Rest:** NeonDB encrypts data using AES-256.
- In Transit:** Mandatory TLS for all API and DB connections.
- Secrets:** All API keys (Clerk, OpenAI, Neon) are strictly stored in Vercel Environment Variables, never committed to Git.

17. DEPLOYMENT ARCHITECTURE

- The project follows a Git-flow CI/CD pipeline hosted on Vercel:
- Push to main branch.
- Vercel triggers build.
- `npx prisma generate` runs to sync types.
- Middleware and Edge functions are deployed.
- NeonDB applies migrations via `prisma migrate deploy`.

18. TESTING STRATEGY

- Integration:** Testing the Clerk Webhook endpoint using ngrok to mock Clerk events.
- Unit:** Validating price calculation logic and scheduling overlap algorithms.
- E2E:** Manual walkthrough of the 'Cook-to-Chef' payment flow.

19. PROJECT TIMELINE & MILESTONES

Milestone	Activity	Duration
M1: Core Infra	Clerk Auth Setup + Prisma Schema Design	Week 1-2
M2: Marketplace	Chef Discovery & User Profile Dashboards	Week 3-5

M3: Intelligence	Riley AI Integration & Real-time Chat	Week 6-7
M4: Production	Payment Logic & Deployment to Vercel	Week 8

20. CHALLENGES FACED & SOLUTIONS

Challenge: Syncing Clerk user roles with the database. **Solution:** Implemented a `/api/webhooks/clerk` route to handle role syncing asynchronously.

Challenge: Handling cold starts in Serverless. **Solution:** Utilized Next.js `stale-while-revalidate` for chef listings to ensure instant page loads.

21. FUTURE ENHANCEMENTS

Visual AI: Allowing Riley to analyze photos of ingredients to suggest recipes.

Subscription Tier: "Modern Kitchen Plus" for unlimited AI chef interactions.

22. CONCLUSION

Modern Kitchen proves that the combination of modern cloud-native tools (Clerk, Prisma, NeonDB) can rapidly bring a complex marketplace to life. By offloading identity and persistence to specialized providers, the team focused on the unique value proposition: connecting humans through culinary art and enhancing the experience with Riley AI. The system is scalable, secure, and ready for high-traffic production.

23. REFERENCES

Clerk Auth Docs: <https://clerk.com/docs>

Prisma Data Models: <https://www.prisma.io/docs>

Neon Serverless Postgres: <https://neon.tech/docs>

Next.js Deployment Docs: <https://nextjs.org/docs>